Documentación Técnica

TP Final · EcoTech Solutions

Normalización de sensores · Catálogo de dispositivos · Panel de sustentabilidad

AUTOR
Marcelo E. Guevara

PROFESOR
Enzo Martín Zotti

INSTITUCIÓN
IPAP

Índice

1. Introducción 3

1.1 Propósito del documento 3

1.2 Componentes del proyecto 3

1.3 Entorno y dependencias 3

1.4 Convenciones de código 3

1.5 Estructura del repositorio 4

1.6 Ejecución rápida 4

2. Ejercicio 1 · Normalización y análisis de sensores 4

2.1 Objetivo 4

2.2 Flujo de datos 4

2.3 Punto A · limpiar_registro_sensor 5

2.4 Punto B · generar_resumen_sensores 6

2.5 Bloque de ejecución y resultados 6

2.6 Decisiones de diseño 6

3. Ejercicio 2 · Catálogo de dispositivos 7

3.1 Objetivo 7

3.2 Jerarquía de clases 7

3.3 StockInvalidoError 7

3.4 DispositivoEcologico (clase base) 7

3.5 PanelSolar (clase derivada) 8

3.6 GestorInventario 8

3.7 Bloque de prueba 9

3.8 Decisiones de diseño 9

4. Ejercicio 3 · Auditoría y panel web 9

4.1 Objetivo 9

4.2 Arquitectura 10

4.3 calcular_emisiones_carbono 10

4.4 obtener_metricas_consumo 10

4.5 Vista y rutas 11

4.6 Plantilla panel.html 11

4.7 Pruebas unitarias · TestCalculoEmisiones 12

5. Validaciones y manejo de errores 12

6. Limitaciones y mejoras posibles 13

1. Introducción

1.1 Propósito del documento

Este documento describe el diseño técnico del TP Final de EcoTech Solutions: qué hace cada componente, cómo está organizado el código, qué contrato tienen las funciones y clases (parámetros, retornos y excepciones), qué decisiones de diseño se tomaron y cómo se verificó el funcionamiento. Las salidas de ejemplo que aparecen en el documento se obtuvieron ejecutando el código del repositorio.

1.2 Componentes del proyecto

Ej.	Módulo principal	Descripción	Tecnologías
1	`Sanitización y Análisis Estadístico de Sensores.py`	Normaliza lecturas térmicas heterogéneas y genera un reporte con alertas y promedio.	Python (biblioteca estándar)
2	`Modelo de Productos, Excepciones y Persistencia.py`	Catálogo de dispositivos ecológicos con validación de stock y exportación a JSON.	Python, POO, `json`
3	`sustentabilidad/services.py` y proyecto Django `ecotech_project`	Cálculo de huella de carbono, pruebas unitarias y panel web de sustentabilidad.	Python, `unittest`, Django

1.3 Entorno y dependencias

Se requiere Python 3.12 o superior. Los ejercicios 1 y 2 usan solo la biblioteca estándar. El ejercicio 3 usa Django para la vista y la plantilla; la lógica de cálculo (`services.py`) y las pruebas unitarias no dependen de Django. Las dependencias están fijadas en `requirements.txt`:

Paquete	Versión	Uso
Django	6.1.3	Framework web del ejercicio 3
asgiref	3.13.0	Dependencia de Django
sqlparse	0.6.1	Dependencia de Django
tzdata	2026.6	Dependencia de Django (datos de zonas horarias)

1.4 Convenciones de código

- Docstrings en estilo Google, con las secciones `Args`, `Returns` y `Raises`.
- Anotaciones de tipos en parámetros, retornos y variables principales.
- Nombres en `snake_case` para funciones y variables, y en `PascalCase` para clases.
- Los errores de datos se comunican con excepciones (`ValueError`, `StockInvalidoError`) y mensajes en español.

1.5 Estructura del repositorio

```
TP Final/
├── Ejercicio 1/
│   └── Sanitización y Análisis Estadístico de Sensores.py
├── Ejercicio 2/
│   └── Modelo de Productos, Excepciones y Persistencia.py
├── Ejercicio 3/
│   ├── manage.py
│   ├── ecotech_project/            # configuración del proyecto Django
│   │   ├── settings.py
│   │   └── urls.py
│   └── sustentabilidad/            # app del panel
│       ├── services.py             # lógica de cálculo
│       ├── views.py                # vista del panel
│       ├── urls.py                 # ruta de la app
│       ├── tests.py                # pruebas unitarias
│       └── templates/sustentabilidad/panel.html
├── requirements.txt
└── README.md
```

1.6 Ejecución rápida

```
# Ejercicios 1 y 2 (desde la carpeta raíz del repositorio)
python "Ejercicio 1/Sanitización y Análisis Estadístico de Sensores.py"
python "Ejercicio 2/Modelo de Productos, Excepciones y Persistencia.py"

# Ejercicio 3 (desde la carpeta "Ejercicio 3")
python manage.py test sustentabilidad
python manage.py runserver        # http://127.0.0.1:8000/sustentabilidad/panel/
```

2. Ejercicio 1 · Normalización y análisis de sensores

Archivo: `Ejercicio 1/Sanitización y Análisis Estadístico de Sensores.py`

2.1 Objetivo

Los microcontroladores de EcoTech transmiten lecturas térmicas con espacios sobrantes, unidades de medida mezcladas con el valor y atributos incompletos. El módulo normaliza cada registro individual y produce un reporte consolidado para la toma de decisiones.

2.2 Flujo de datos

Etapa	Componente	Entrada → Salida
1	`limpiar_registro_sensor`	Registro crudo (`dict`) → registro normalizado (`dict`) o `ValueError`
2	Bucle con `try/except`	Registros válidos se acumulan en una lista; los inválidos se informan y se descartan
3	`generar_resumen_sensores`	Lista de registros limpios + umbral → tupla `(ids_en_alerta, promedio)`
4	Impresión del reporte	Resultados → texto formateado en consola

Estructuras de datos

Estructura	Claves	Detalle
Registro crudo	`sensor_id`, `temperatura`, `ubicacion`	Textos con espacios y unidades; `ubicacion` puede faltar o venir vacía.
Registro limpio	`sensor_id`, `temperatura`, `ubicacion`	`str`, `float` y `str` respectivamente; la ubicación nunca queda vacía.

2.3 Punto A · limpiar_registro_sensor

```
limpiar_registro_sensor(registro_raw: dict[str, str])
```

Elemento	Descripción
Parámetro	`registro_raw`: diccionario con los datos crudos de un sensor.
Retorna	Un **nuevo** diccionario con `sensor_id` (str), `temperatura` (float) y `ubicacion` (str). El original no se modifica.
Excepciones	`ValueError` si falta el `sensor_id` o si la temperatura no es interpretable.

Algoritmo

- **Identificador:** se lee con `.get("sensor_id", "")`, se convierte a texto y se aplica `.strip()`. Si queda vacío se lanza `ValueError("falta el sensor_id")`.
- **Temperatura:** se recorre el texto carácter a carácter y se conservan solo los dígitos, `.`, `,` y `-`. La coma se reemplaza por punto y se convierte con `float()`. Si no queda ningún número válido, se lanza `ValueError` indicando el sensor y el texto recibido.
- **Ubicación:** si la clave no existe, es `None` o queda vacía tras `.strip()`, se asigna el valor por defecto «Desconocida».
- **Retorno:** se arma y devuelve un diccionario nuevo con los tres campos.

Transformaciones (ejemplos verificados)

Campo	Valor crudo	Resultado
sensor_id	`" SENSOR-001 "`	`"SENSOR-001"`
temperatura	`"24.5C"`	`24.5`
temperatura	`" 31.2 °C "`	`31.2`
temperatura	`"28,7C"` (coma decimal)	`28.7`
temperatura	`"0 C"`	`0.0` (lectura válida)
ubicacion	`" Planta Sur "`	`"Planta Sur"`
ubicacion	clave ausente o `""`	`"Desconocida"`

Casos de error

Entrada	Resultado
`sensor_id` ausente o compuesto solo por espacios	`ValueError: falta el sensor_id`
`temperatura` sin dígitos (`"error"`, `""`)	`ValueError: temperatura inválida en <id>: <texto>`
`temperatura` mal formada (`"1.2.3"`, `"-"`)	Mismo `ValueError`, porque `float()` falla

2.4 Punto B · generar_resumen_sensores

```
generar_resumen_sensores(lista_sensores_limpios: list[dict],
                         umbral_temperatura: float) -> tuple[list[str], float]
```

Elemento	Descripción
Parámetros	`lista_sensores_limpios`: registros ya normalizados. `umbral_temperatura`: límite térmico.
Retorna	Tupla `(lista_sensores_alerta, promedio_temperatura)`: IDs de los sensores cuya temperatura supera el umbral y promedio global.

Algoritmo

- Si la lista está vacía devuelve `([], 0.0)`, lo que evita la división por cero.
- Recorre la lista una sola vez: acumula la suma de temperaturas y agrega el ID a la lista de alertas cuando `temperatura > umbral` (comparación estricta).
- El promedio es la suma dividida por la cantidad de registros válidos. Complejidad lineal, O(n).

2.5 Bloque de ejecución y resultados

El bloque final procesa siete registros crudos con un umbral de 30.0 °C. Cada registro pasa por `limpiar_registro_sensor`; si lanza `ValueError`, se imprime una advertencia y se continúa con el siguiente.

Registro	Entrada relevante	Resultado
SENSOR-001	`" SENSOR-001 "`, `"24.5C"`	24.5 °C · Planta Norte · OK
SENSOR-002	`" 31.2 °C "`, ubicación con espacios	31.2 °C · Planta Sur · **ALERTA**
SENSOR-003	`"28,7C"`, sin ubicación	28.7 °C · Desconocida · OK
SENSOR-004	`"35.0C"`, ubicación vacía	35.0 °C · Desconocida · **ALERTA**
SENSOR-005	`"error"`	Descartado con advertencia
SENSOR-006	`"22C"`	22.0 °C · Planta Norte · OK
SENSOR-007	`"0 C"`, ubicación vacía	0.0 °C · Desconocida · OK

Salida real del programa:

```
[ADVERTENCIA] Registro descartado: temperatura inválida en SENSOR-005: 'error'

=== REPORTE TÉRMICO - ECOTECH SOLUTIONS ===
Registros válidos: 6 de 7
Umbral de alerta : 30.0 °C
Promedio global  : 23.57 °C
-----------------------------------------
SENSOR-001    24.5 °C  Planta Norte   OK
SENSOR-002    31.2 °C  Planta Sur     ALERTA
SENSOR-003    28.7 °C  Desconocida    OK
SENSOR-004    35.0 °C  Desconocida    ALERTA
SENSOR-006    22.0 °C  Planta Norte   OK
SENSOR-007     0.0 °C  Desconocida    OK
-----------------------------------------
Sensores en alerta: SENSOR-002, SENSOR-004
```

2.6 Decisiones de diseño

- **Función pura:** la limpieza devuelve un diccionario nuevo y no modifica el registro original.

- **Tolerancia a datos incompletos:** `.get()` con valor por defecto evita `KeyError` cuando faltan atributos.
- **El cero es un dato válido:** `"0 C"` se interpreta como 0.0 y entra en el promedio. Solo los registros que no se pueden interpretar se descartan.
- **Promedio sobre registros válidos:** los registros descartados no participan del cálculo (6 registros en el ejemplo: 141.4 / 6 = 23.57).
- **Umbral estricto:** un sensor entra en alerta solo si su temperatura es mayor que el umbral; un valor igual al umbral no alerta.

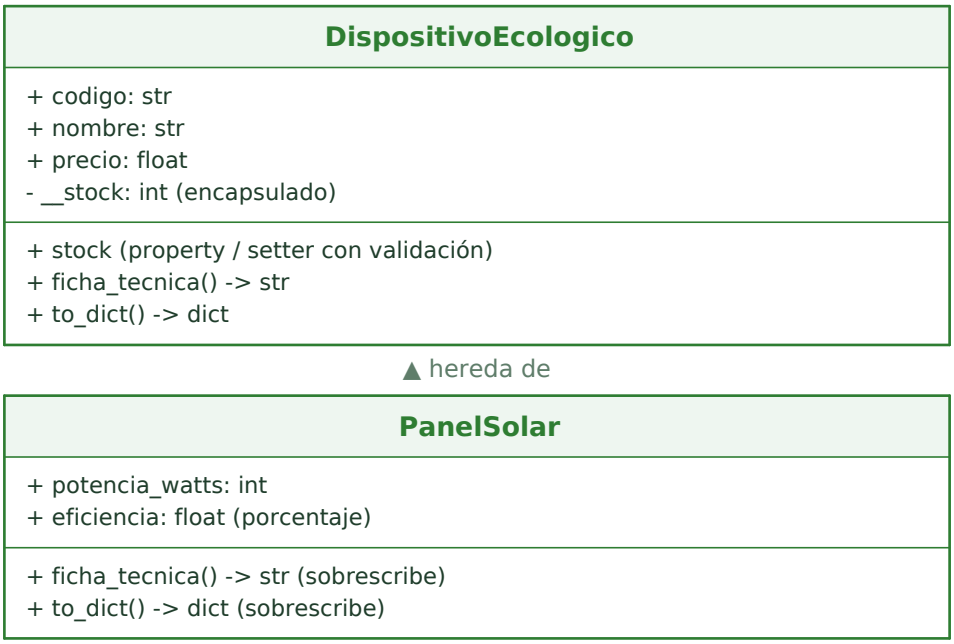
3. Ejercicio 2 · Catálogo de dispositivos

Archivo: `Ejercicio 2/Modelo de Productos, Excepciones y Persistencia.py`

3.1 Objetivo

Gestionar los productos del catálogo de equipamiento sustentable, impedir inconsistencias de inventario mediante validaciones de negocio y persistir el catálogo en disco en formato JSON.

3.2 Jerarquía de clases



Además: `StockInvalidoError` *hereda de* `Exception` *y es lanzada por* `DispositivoEcologico`*.* `GestorInventario` *recibe una lista de* `DispositivoEcologico` *(o sus subclases) para persistirla.*

3.3 StockInvalidoError

Excepción de dominio que hereda de `Exception`. No agrega lógica propia: su valor está en tener un nombre específico del negocio, lo que permite capturarla con `except StockInvalidoError` sin atrapar otros errores. Se lanza cuando se intenta asignar un stock negativo.

3.4 DispositivoEcologico (clase base)

Encapsulamiento del stock

El stock se guarda en un atributo con doble guion bajo (`__stock`). Python aplica «name mangling»: internamente se almacena como `_DispositivoEcologico__stock`, lo que impide el acceso accidental desde fuera de la clase. La única vía de acceso es la propiedad `stock`:

- **Getter** (`@property`): devuelve el valor actual.
- **Setter** (`@stock.setter`): si el valor es menor que 0 lanza `StockInvalidoError` con un mensaje que incluye el nombre del producto y el valor rechazado; si no, lo asigna.
- **Constructor:** ejecuta `self.stock = stock` (usa el setter y no el atributo privado), de modo que la validación también se aplica al crear el objeto: no se puede instanciar un dispositivo con stock negativo.

Métodos

Método	Descripción
`__init__(codigo, nombre, precio, stock)`	Inicializa los atributos y valida el stock inicial. Lanza `StockInvalidoError` si es negativo.
`stock` (getter)	Devuelve el stock actual (`int`).
`stock` (setter)	Asigna un nuevo stock. Lanza `StockInvalidoError` si el valor es negativo.
`ficha_tecnica() -> str`	Ficha comercial en dos líneas: código y nombre, y luego precio y stock.
`to_dict() -> dict`	Diccionario con `tipo` (nombre de la clase), `codigo`, `nombre`, `precio` y `stock`, listo para serializar.

3.5 PanelSolar (clase derivada)

Hereda de `DispositivoEcologico` y agrega `potencia_watts` (int) y `eficiencia` (float, en porcentaje). Su constructor llama a `super().__init__(...)`, por lo que la validación de stock se aplica igual que en la clase base.

Polimorfismo

- `ficha_tecnica()` se sobrescribe: reutiliza la ficha base con `super().ficha_tecnica()` y le agrega una tercera línea con la potencia y la eficiencia.
- `to_dict()` se sobrescribe: toma el diccionario base y suma `potencia_watts` y `eficiencia`. El campo `tipo` refleja la clase real (`PanelSolar`) porque se obtiene con `self.__class__.__name__`.
- Quien llama usa siempre `dispositivo.ficha_tecnica()` o `dispositivo.to_dict()` sin importar el tipo concreto; cada objeto responde con su propia versión.

```
[PS-001] Panel Solar 300W
    Precio: $45000.00 | Stock: 10 unidades
    Potencia: 300 W | Eficiencia: 18.5%
```

3.6 GestorInventario

```
guardar_catalogo_json(self, lista_dispositivos: list[DispositivoEcologico],
                      ruta_archivo: str)
```

Elemento	Descripción
Parámetros	`lista_dispositivos`: dispositivos a guardar. `ruta_archivo`: ruta del archivo de salida (por ejemplo `catalogo_ecotech.json`).
Proceso	Convierte cada objeto con `to_dict()`, abre el archivo en modo escritura con codificación UTF-8 y vuelca la lista con `json.dump(..., indent=4, ensure_ascii=False)`.
Efecto	Crea el archivo o lo sobrescribe si ya existe. `ensure_ascii=False` conserva los acentos legibles.

Formato del archivo generado (lista de objetos, uno por dispositivo):

```
[
    {
        "tipo": "PanelSolar",
        "codigo": "PS-001",
        "nombre": "Panel Solar 300W",
        "precio": 45000.0,
        "stock": 10,
        "potencia_watts": 300,
        "eficiencia": 18.5
    }
]
```

3.7 Bloque de prueba

- Se instancia un `PanelSolar` con stock 10 y se imprime su ficha técnica.
- Dentro de un `try/except StockInvalidoError` se intenta asignar `stock = -5`; la excepción se captura y se imprime su mensaje.
- Se crea un `GestorInventario`, se arma una lista con el panel (que conserva su stock válido de 10) y se exporta a `catalogo_ecotech.json`.

Salida real del programa:

```
[PS-001] Panel Solar 300W
    Precio: $45000.00 | Stock: 10 unidades
    Potencia: 300 W | Eficiencia: 18.5%

Intentando asignar stock negativo...
Excepción capturada correctamente: Stock inválido para 'Panel Solar 300W': -5 (no puede ser negativo)

Catálogo guardado exitosamente en 'catalogo_ecotech.json'.
```

3.8 Decisiones de diseño

- **Validación centralizada:** todo cambio de stock pasa por el setter, por lo que la regla «el stock no puede ser negativo» vive en un único lugar.
- **Excepción propia:** `StockInvalidoError` distingue un error de negocio de errores genéricos de Python.
- **Serialización delegada:** cada clase sabe convertirse a diccionario (`to_dict`), por lo que el gestor funciona con cualquier subclase futura sin modificarse.
- **Extensibilidad:** un nuevo tipo de producto solo necesita heredar de `DispositivoEcologico` y sobrescribir `ficha_tecnica()` y `to_dict()`.

4. Ejercicio 3 · Auditoría y panel web

4.1 Objetivo

Certificar con pruebas automatizadas el algoritmo de cálculo de huella de carbono industrial y luego integrarlo en la tubería de datos que alimenta el panel web administrativo, desarrollado con Django.

4.2 Arquitectura

Archivo	Responsabilidad
`sustentabilidad/services.py`	Lógica de negocio: `calcular_emisiones_carbono` y `obtener_metricas_consumo`. No depende de Django.
`sustentabilidad/views.py`	`vista_panel_sustentabilidad` y los datos de ejemplo `HISTORIAL_CONSUMOS`.
`sustentabilidad/urls.py`	Define la ruta `panel/` de la app.
`ecotech_project/urls.py`	Ruta raíz del proyecto: `admin/` y el prefijo `sustentabilidad/` que incluye las URLs de la app.
`sustentabilidad/templates/sustentabilidad/panel.html`	Plantilla HTML del panel.
`sustentabilidad/tests.py`	Pruebas unitarias: clase `TestCalculoEmisiones`.
`ecotech_project/settings.py`	Registra la app en `INSTALLED_APPS`; `APP_DIRS = True` permite encontrar las plantillas dentro de la app.

Recorrido de una petición a `/sustentabilidad/panel/`:

Paso	Componente	Acción
1	`ecotech_project/urls.py`	Reconoce el prefijo `sustentabilidad/` y delega en `sustentabilidad.urls`.
2	`sustentabilidad/urls.py`	Reconoce `panel/` y llama a `vista_panel_sustentabilidad`.
3	`views.py`	Invoca `obtener_metricas_consumo(HISTORIAL_CONSUMOS)` y arma el diccionario de contexto.
4	`services.py`	Calcula las emisiones por planta, el total global y la planta de mayor consumo.
5	`render(...)`	Combina `panel.html` con el contexto y devuelve la respuesta HTML.

4.3 calcular_emisiones_carbono

```
calcular_emisiones_carbono(consumo_kwh: float, factor_emision: float = 0.4) -> float
```

Elemento	Descripción
Fórmula	`emisiones = consumo_kwh * factor_emision`, con el factor expresado en kg de CO_2 por kWh.
Parámetros	`consumo_kwh`: consumo en kWh (no puede ser negativo). `factor_emision`: opcional, 0.4 por defecto.
Retorna	Emisiones de carbono asociadas al consumo (`float`).
Excepciones	`ValueError` si `consumo_kwh < 0`. El valor 0 es válido y produce 0 emisiones.

4.4 obtener_metricas_consumo

```
obtener_metricas_consumo(registros: list[dict]) -> dict
```

Recibe un historial de consumos con la forma `{"planta": "Planta Norte", "consumo_kwh": 1200.0}`. Una misma planta puede aparecer varias veces; sus consumos se acumulan.

Algoritmo

- Por cada registro calcula las emisiones con `calcular_emisiones_carbono` (factor por defecto) y acumula, por planta, el consumo en kWh y las emisiones. Usa `.get(planta, 0.0)` para iniciar una planta la primera vez que aparece.
- **Planta de mayor consumo:** se determina con `max` sobre el consumo acumulado en kWh. En caso de empate gana la primera encontrada. Si no hay registros, el valor es `None`.
- **Total global:** suma de las emisiones acumuladas de todas las plantas.

Diccionario devuelto

Clave	Tipo	Contenido
`total_emisiones`	float	Emisiones globales de todas las plantas.
`planta_mayor_consumo`	str | None	Nombre de la planta con más consumo acumulado.
`emisiones_por_planta`	dict[str, float]	Emisiones acumuladas de cada planta.

Resultado con los datos de ejemplo (`HISTORIAL_CONSUMOS`):

Planta	Registros	Consumo acumulado	Emisiones (kg CO_2)
Planta Norte	1200.0 + 300.0 kWh	1500.0 kWh	600.00
Planta Sur	950.5 kWh	950.5 kWh	380.20
Planta Este	780.0 kWh	780.0 kWh	312.00
Total global		3230.5 kWh	**1292.20**

*La planta de mayor consumo es **Planta Norte**. Este es el mismo resultado que se muestra en el panel.*

4.5 Vista y rutas

```
vista_panel_sustentabilidad(request: HttpRequest) -> HttpResponse
```

La vista procesa el historial con `obtener_metricas_consumo` y renderiza `sustentabilidad/panel.html` con el siguiente contexto:

Variable de contexto	Origen	Uso en la plantilla
`titulo`	Texto fijo de la vista	Título de la página y encabezado.
`total_emisiones`	`metricas["total_emisiones"]`	Tarjeta con el total global.
`planta_mayor_consumo`	`metricas["planta_mayor_consumo"]`	Tarjeta destacada.
`emisiones_por_planta`	`metricas["emisiones_por_planta"]`	Filas de la tabla por planta.

```
# ecotech_project/urls.py
path('admin/', admin.site.urls),
path("sustentabilidad/", include("sustentabilidad.urls"))

# sustentabilidad/urls.py
path("panel/", vista_panel_sustentabilidad, name="panel_sustentabilidad")
```

La URL final del panel es `/sustentabilidad/panel/`. Los datos de `HISTORIAL_CONSUMOS` son de ejemplo; en un sistema real provendrían de la base de datos.

4.6 Plantilla panel.html

- `{{ total_emisiones|floatformat:2 }}`: muestra el total con dos decimales (por ejemplo 1292.20) y oculta el ruido de punto flotante.

- `{{ planta_mayor_consumo|default:"Sin datos" }}`: si no hay plantas, muestra «Sin datos» en lugar de un valor vacío.
- `{% for planta, emisiones in emisiones_por_planta.items %}`: recorre el diccionario para generar una fila por planta.
- **Diseño:** encabezado con degradé verde, dos tarjetas de métricas, tabla de emisiones con filas alternadas y una columna lateral con una ilustración SVG de bienvenida. Los estilos CSS están dentro del propio archivo.
- **Comportamiento responsivo:** la columna lateral queda fija al desplazarse la página y se oculta en pantallas de 900 px de ancho o menos, para no tapar el contenido.

4.7 Pruebas unitarias · TestCalculoEmisiones

La clase hereda de `unittest.TestCase` y verifica tanto la exactitud numérica como la elevación de la excepción. Para comparar decimales usa `assertAlmostEqual`, que tolera las diferencias de precisión del punto flotante.

Prueba	Entrada	Resultado esperado
`test_calculo_con_factor_por_defecto`	`100`	`40.0`
`test_calculo_con_factor_personalizado`	`250`, factor `0.5`	`125.0`
`test_calculo_con_decimales`	`12.5`	`5.0`
`test_consumo_cero`	`0`	`0.0` (valor límite válido)
`test_consumo_negativo_lanza_error`	`-10`	`ValueError`
`test_consumo_negativo_con_factor_personalizado`	`-1`, `factor_emision=0.8`	`ValueError`

Resultado de la ejecución:

```
Ran 6 tests in 0.000s

OK
```

Comando: `python manage.py test sustentabilidad`, desde la carpeta `Ejercicio 3`. El ejecutor de pruebas de Django descubre las clases basadas en `unittest.TestCase`.

5. Validaciones y manejo de errores

Resumen de las validaciones de negocio del proyecto y de cómo responde cada componente.

Ej.	Situación	Mecanismo	Resultado
1	Registro sin `sensor_id` válido	`raise ValueError`	El bucle principal informa y descarta el registro.
1	Temperatura sin número interpretable	`float()` falla y se relanza `ValueError`	Mensaje con el sensor y el texto recibido; el registro se descarta.
1	Ubicación ausente o vacía	Valor por defecto	«Desconocida»; el registro se conserva.
1	Lista vacía en el resumen	Retorno anticipado	`([], 0.0)`, sin división por cero.
2	Stock negativo (creación o asignación)	Setter de `stock`	`StockInvalidoError` con nombre del producto y valor rechazado.
3	Consumo negativo	`raise ValueError` en `calcular_emisiones_carbono`	Se propaga a `obtener_metricas_consumo` y a la vista.
3	Historial sin registros	`if consumo_por_planta`	Total 0 y planta de mayor consumo `None`; la plantilla muestra «Sin datos».

6. Limitaciones y mejoras posibles

Puntos identificados durante la revisión del código que no afectan el funcionamiento con los datos de prueba, pero que se pueden reforzar en una próxima iteración.

Ej.	Observación	Mejora sugerida
1	El flujo de prueba está a nivel de módulo, sin `if __name__ == "__main__":`; si el archivo se importa, el reporte se ejecuta.	Encapsularlo en una función `main()` protegida por ese condicional.
1	La extracción de temperatura concatena todos los dígitos del texto, por lo que asume un único valor numérico por campo.	Extraer solo el primer número (por ejemplo con una expresión regular).
1	Un `sensor_id` presente con valor `None` se convierte en el texto «None» en lugar de rechazarse.	Usar `registro_raw.get("sensor_id") or ""` antes de `str()`.
2	El setter valida que el stock no sea negativo, pero no su tipo ni el precio.	Validar tipo numérico y precio no negativo.
2	El bloque de prueba verifica la asignación negativa, pero no la creación de un objeto con stock negativo (que también se rechaza).	Agregar ese caso a las pruebas.
3	`obtener_metricas_consumo` no tiene pruebas propias; se verificó manualmente con los datos de ejemplo.	Agregar una clase de pruebas para total, planta de mayor consumo y lista vacía.
3	Los consumos están fijos en `views.py`; `models.py` está vacío.	Definir un modelo y leer el historial desde la base de datos.
3	`max(..., key=consumo_por_planta.get)` funciona, pero el analizador de tipos del editor lo marca.	Usar `key=lambda planta: consumo_por_planta[planta]`.
3	La configuración es de desarrollo (`DEBUG = True`, SQLite).	Ajustarla antes de un despliegue real.